Chapter 14. Network Clients

14.0 Introduction

Java can be used to write many types of networked programs. In traditional socket-based code, the programmer is responsible for structuring the interaction between the client and server; the TCP *socket code* simply ensures that whatever data you send gets to the other end. In higher-level types, such as HTTP, RMI, CORBA, and EJB, the software takes over more control. Sockets are often used for connecting to legacy servers; if you were writing a new application from scratch, you'd be better off using a higher-level service.

It may be helpful to compare sockets with the telephone system. Telephones were originally used for analog voice traffic, which is pretty unstructured. Then they began to be used for some layered applications; the first widely popular one was facsimile transmission, or fax. Where would fax have been without the widespread availability of voice telephony? The second wildly popular layered application historically was dial-up TCP/IP. This coexisted with the web to become popular as a mass-market service. Where would dial-up IP be without widely deployed voice lines? And where would the internet be without dial-up IP? Fax and dial-up are mostly gone now, but they paved the way for your smartphone's networked ability, which is what makes it useful (and even seductive as a time sink).

Sockets are layered like that too. The web, RMI, JDBC, CORBA, and EJB are all layered on top of sockets. HTTP is now the most common protocol and should generally be used for new applications when all you want is to get data from point b to point a. Figure 14-1 shows a very simplified diagram of network layers.

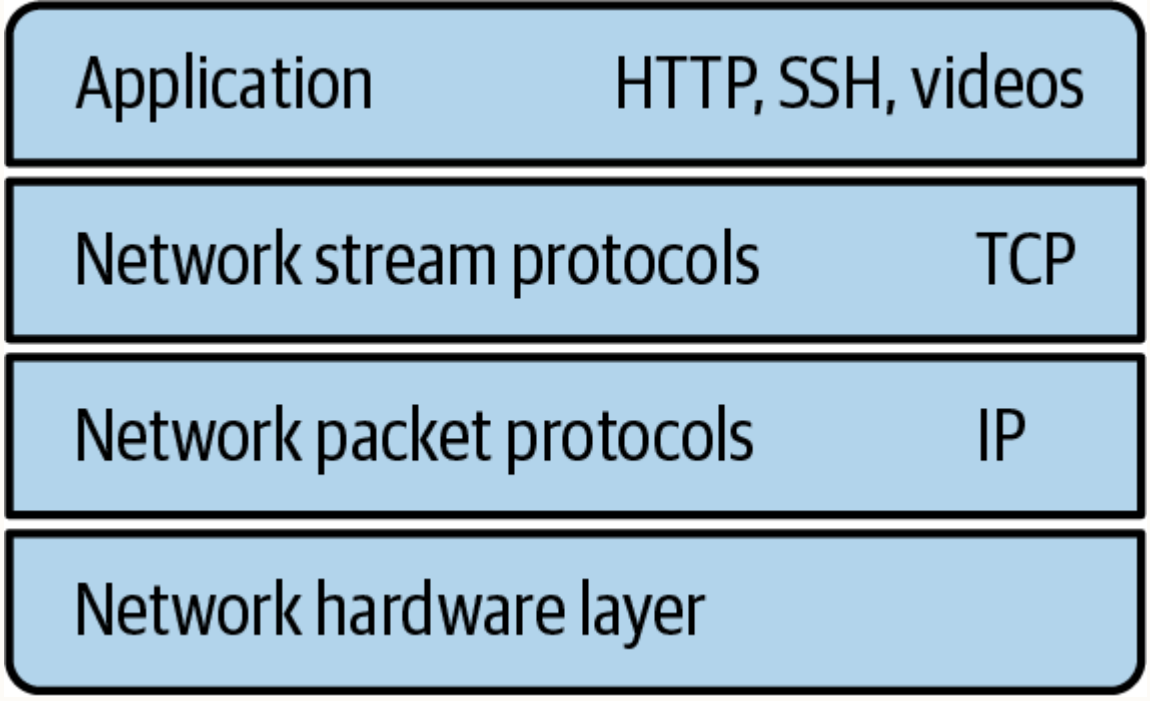


Figure 14-1. Network layers (simplified)

Ever since the alpha release of Java (originally as a sideline to the HotJava browser) in May 1995, Java has been popular as a programming language for building network applications. It's easy to see why, particularly if you've ever built a networked application in C. First, C programmers have to worry about the platform they are on. Unix uses synchronous sockets, which work rather like normal disk files vis-à-vis reading and writing, whereas Microsoft OSes use asynchronous sockets, which use callbacks to notify when a read or write has completed. Java glosses over this distinction. Further, the amount of code needed to set up a socket in C is intimidating. Just for fun, Example 14-1 shows the typical C code for setting up a client socket. Remember, this is only the Unix part. And only the part that makes and closes the connection. To be portable to Windows, it would need some additional conditional code (using C's `#ifdef` mechanism). C's `#include` mechanism requires that exactly the right files be included, and some files have to be listed in a particular order (Java's `import` mechanism is much more flexible).

Example 14-1. main/src/main/java/network/Connect.c (C client setup)

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>
#include <stdio.h>
#include <string.h>
#include <fcntl.h>

int
main(int argc, char *argv[])
{
```

```
  char* server_name = "localhost";
  struct hostent *host_info;
  int sock;
  struct sockaddr_in server;

  /* Look up the remote host's IP address */
  host_info = gethostbyname(server_name);
  if (host_info == NULL) {
    fprintf(stderr, "%s: unknown host: %s\n", argv[0], server_name);
    exit(1);
  }

  /* Create the socket */
  if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
    perror("creating client socket");
    exit(2);
  }

  /* Set up the server's socket address */
  server.sin_family = AF_INET;
  memcpy((char *)&server.sin_addr, host_info->h_addr,
          host_info->h_length);
  server.sin_port = htons(80);

  /* Connect to the server */
  if (connect(sock,(struct sockaddr *)&server,sizeof server) < 0) {
    perror("connecting to server");
    exit(4);
  }

  /* Finally, we can read and write on the socket. */
  /* ... */

  (void) close(sock);
}
```

Java replaces all of that setup code (up to the read/write/close) with:

```
Socket s = new Socket("localhost", 80);
```

In an early recipe, we'll see how to do the connect in this one line of Java (plus a bit of error handling). We'll then cover error handling and transferring data over a socket. Next, we'll take a quick look at a `datagram` or UDP client that implements most of the

Trivial File Transfer Protocol (TFTP) that has been used for two decades to boot diskless workstations. We'll end with a program that connects interactively to a chat server.

A common theme through most of these client examples is to use existing servers so that we don't have to generate both the client and the server at the same time. Most of these are services that exist on any standard Unix platform. If you can't find a Unix server near you to try them on, let me suggest that you take an old PC, maybe one that's underpowered for running the latest Microsoft software, and put up a free, open source Unix system on it. My personal favorite is OpenBSD, and the market's overall favorite is Linux. Both are readily available, can be installed for free over the internet, and offer all the standard services used in the client examples, including the time servers and TFTP. Java runs on all of them.

I also provide basic coverage of web services clients. The term *web services* has come to mean program-to-program communication using HTTP. The two general categories are SOAP-based and REST-based. REST services are very simple—you send an HTTP request and get back a response in plain text, JSON (Chapter 16), or XML. SOAP is more complicated and not covered in this book. There is more information on the client-side connections in *Java Network Programming* by Elliotte Rusty Harold (O'Reilly). I don't cover the server-side APIs for building web services—JAX-RS and JAX-WS—because these are covered in several O'Reilly books.

14.1 HTTP/REST Web Client—Modern API 11

Problem

You need to read from a URL, for example, to connect to a RESTful web service or to download a web page or other resource over HTTP/HTTPS.

Solution

Use the standard Java 11 `HttpClient` or the `URLConnection` class.

This technique applies anytime you need to read from a URL, not just when you need to connect to a RESTful web service.

Discussion

Prior to Java 11, you had to either use the `HttpURLConnection` class or download and use the older Apache HttpClient library. With Java 11, there is a fairly easy-to-use and flexible API in standard Java. The `HttpURLConnection` is also still available.

As our basic example, we'll use Google's Suggest service, that is, what you see when you type the first few characters of a search term into the Google web search engine. This Google service supports various output formats. The base URL is the following:

```
https://suggestqueries.google.com/complete/search?client=firefox&q=
```

Append to it the word for which you want suggestions. The `client=firefox` tells it we want a simple JSON format; with `client=chrome` it contains more fields.

To use the Java HTTP Client API, you need an `HttpClient` object, which you get using the Builder pattern. You then create a `Request` object, as in Example 14-2.

Example 14-2. main/src/main/java/netweb/HttpClientDemo—Create client and request

```
// This object would be kept for the life of an application
HttpClient client = HttpClient.newBuilder()
 .followRedirects(Redirect.NORMAL)
 .version(Version.HTTP_1_1)
 .build();

// Build the HttpRequest object to "GET" the urlString
HttpRequest req =
 HttpRequest.newBuilder(URI.create(urlString +
  URLEncoder.encode(input, StandardCharsets.UTF_8)))
 .header("User-Agent", "Ministry of Silly Walks")
 .GET()
 .build();
```

The `HttpRequest` object can be sent using the client to get an `HttpResponse` object, from which you can get the status and/or the body. Sending can be done either synchronously (if you need the results right away) or asynchronously (if you can usefully do something else in the meantime). This example shows sending it both synchronously and asynchronously:

```
    // Send the request - synchronously
    HttpResponse<String> resp =
     client.send(req, BodyHandlers.ofString());

    // Collect the results
    if (resp.statusCode() == 200) {
     String response = resp.body();
     System.out.println(response);
    } else {
     System.out.printf("ERROR: Status %d on request %s\n",
      resp.statusCode(), urlString);
    }
```

```
    // Send the request - asynchronously
    client.sendAsync(req, BodyHandlers.ofString())
     .thenApply(HttpResponse::body)
     .thenAccept(System.out::println)
     .join();
```

Here is the output (given the input is set to `"darwin"`); the line has been broken at commas to make it fit on the page:

```
$ java HttpClientDemo.java
["darwin",["darwin thompson","darwin","darwin awards","darwinism",
 "darwin australia","darwin thompson fantasy","darwin barney",
 "darwin theory","darwinai","darwin dormitorio"]]
```

There are times when you might not want to use the HttpClient library. You may be stuck on a very old version of Java. Or you may be familiar with the legacy code and decide that you don't need the features that the new API provides. In such cases you could use the legacy code in `java.net`, since all that's usually needed here is the ability to open and read from a URL. Here is similar code using a `URLConnection` (actually an `HttpURLConnection`):

```
public class RestClientURLDemo {
 public static void main(String[] args) throws Exception {
  HttpURLConnection conn = (HttpURLConnection)
   URI.create(HttpClientDemo.urlString +
    HttpClientDemo.keyword).toURL()
    .openConnection();
```

```
    try (BufferedReader is =
      new BufferedReader(
        new InputStreamReader(conn.getInputStream()))) {
      String line;
      while ((line = is.readLine()) != null) {
        System.out.println(line);
      }
    }
  }
}
```

The output should be identical to what the `HttpClient` version produced.

See Also

Don't confuse this `HttpClient` with the older Apache HttpClient library.

You can find more information on REST services (including implementing the server-side components for them) in Bill Burke's *RESTful Java with JAX-RS 2.0* (O'Reilly).

14.2 Contacting a Socket Server

Problem

You need to contact a server using TCP/IP.

Solution

Just create a `java.net.Socket`, passing the hostname and port number into the constructor.

Discussion

There isn't much to this in Java. When creating a socket, you pass in the hostname and the port number. The `java.net.Socket` constructor does the `gethostbyname()` and the `socket()` system call, sets up the server's `sockaddr_in` structure, and executes the `connect()` call. All you have to do is catch the errors, which are subclassed from the familiar `IOException`. Example 14-3 sets up a Java network client and does the absolute bare minimum to replicate the Google Suggest service from Example 14-2. It uses `try-`

`with-resources` to ensure that the socket and the two streams are closed automatically when we are done.

Example 14-3. main/src/main/java/network/ConnectSimple.java (simple client connection)

```
import java.net.Socket;
import java.io.*;

/* Client with NO error handling */
public class ConnectSimple {

  public static void main(String[] argv) throws Exception {

    try (Socket sock = new Socket("suggestqueries.google.com", 80);
      PrintStream pout = new PrintStream(sock.getOutputStream());
      BufferedReader is =
        new BufferedReader(new InputStreamReader(sock.getInputStream())) {

      pout.println("GET /complete/search?client=firefox&q=darwin HTTP/1.0\r");
      pout.println("\r");

      String line;
      while ((line = is.readLine()) != null) {
        System.out.println(line);
      }
    }
  }
}
```

This version does no real error reporting, but a version called `ConnectFriendly` does; we'll see this version in Recipe 14.4.

See Also

Java supports other ways of using network applications. You can also open a URL and read from it (see Recipe 14.8). You can write code so that it will run from a URL, when opened in a web browser, or from an application.

14.3 Finding and Reporting Network Addresses

Problem

You want to look up a host's address name or number or get the address at the other end of a network connection.

Solution

Get an `InetAddress` object.

Discussion

The `InetAddress` object represents the internet address of a given computer or host. It has no public constructors; you obtain an `InetAddress` by calling the static `getByName()` method, passing in either a hostname like *darwinsys.com* or a network address as a string, like 1.23.45.67. All the lookup methods in this class can throw the checked `UnknownHostException` (a subclass of `java.io.IOException`), which must be caught or declared on the calling method's definition. None of these methods actually contact the remote host, so they do not throw the other exceptions related to network connections.

The method `getHostAddress()` gives you the numeric IP address (as a string) corresponding to the `InetAddress`. The inverse is `getHostName()`, which reports the name of the `InetAddress`. This can be used to print the address of a host given its name, or vice versa:

```
public class InetAddrDemo {
  public static void main(String[] args) throws IOException {
    String hostName = "darwinsys.com";
    String ipNumber = "8.8.8.8"; // currently a well-known Google DNS server

    // Show getting the InetAddress (looking up a host) by host name
    System.out.println(hostName + "'s address is " +
      InetAddress.getByName(hostName).getHostAddress());

    // Look up a host by address
    System.out.println(ipNumber + "'s name is " +
      InetAddress.getByName(ipNumber).getHostName());
```

```
    // Look up my localhost addresss
    final InetAddress localHost = InetAddress.getLocalHost();
    System.out.println("My localhost address is " + localHost);

    // Show getting the InetAddress from an open Socket
    String someServerName = "google.com";
    // assuming there's a web server on the named server:
    try (Socket theSocket = new Socket(someServerName, 80)) {
     InetAddress remote = theSocket.getInetAddress();
     System.out.printf("The InetAddress for %s is %s%n",
      someServerName, remote);
    }
   }
}
```

You can also get an `InetAddress` from a `Socket` by calling its `getInetAddress()` method. You can construct a `Socket` using an `InetAddress` instead of a hostname string. So, to connect to port number `myPortNumber` on the same host as an existing socket, use this:

```
InetAddress remote = theSocket.getInetAddress( );
Socket anotherSocket = new Socket(remote, myPortNumber);
```

Finally, to look up all the addresses associated with a host—a server may be on more than one network—use the static method `getAllByName(host)`, which returns an array of `InetAddress` objects, one for each IP address associated with the given name.

A static method `getLocalHost()` returns an `InetAddress` equivalent to `localhost` or 127.0.0.1. This can be used to connect to a server program running on the same machine as the client.

The number of all possible IPv4 addresses was sufficient back in the day when the research-based ARPANET gave way to the internet. However, once consumers, and devices like smartphones, came along, the number of devices far exceeded the number of addresses. Two solutions exist:

- Network Address Translation (NAT), which gives computers inside your router or firewall addresses drawn from a "private" pool, and translates them into a public address as packets come and go.
- IPv6, the sixth incarnation of the Internet Protocol (IP), which has a 128-bit address space, whose theoretical range exceeds by 22 orders of magnitude the number of

smartphones it would take to completely cover all the land on this planet.[1]

If you want IPv6 addresses, you can use `Inet6Address` instead. There is a `getAllByName()` in `InetAddress` and its two subclasses, `Inet4Address` and `Inet6Address`. Note that if your computer is configured for both, `getAllByName()` might return both IPv4 and IPv6 addresses:

```
$ jshell
> import java.net.*;
> Inet6Address.getAllByName("google.com");
$1 ==> InetAddress[2] { google.com/142.250.190.110,
      google.com/2607:f8b0:4009:809:0:0:0:200e }
```

See Also

See `NetworkInterface` in Recipe 15.2, which lets you find out more about the networking of the machine you are running on. There is no way to look up services in the standard API yet—that is, to find out that the HTTP service is on port 80. Full implementations of TCP/IP have always included an additional set of resolvers; in C, the call `getservbyname("http", "tcp");` would look up the given service[2] and return a `servent` (service entry) structure whose `s_port` member would contain the value 80. The numbers of established services do not change, but when services are new or installed in nonroutine ways, it is convenient to be able to change the service number for all programs on a machine or network (regardless of programming language) just by changing the services definitions. Java should provide this capability in a future release.

14.4 Handling Network Errors

Problem

There are many things that can go wrong during a network interaction. You want more detailed reporting than just `IOException` if something goes wrong.

Solution

Catch a greater variety of exception classes, and report them separately.

Discussion

`SocketException` has several subclasses; the most notable are `ConnectException` and `NoRouteToHostException`. The names are mostly self-explanatory: the first means that the connection was refused by the machine at the other end (the server machine has no service running on the given TCP port), and the second completely explains the failure. Example 14-4 is an excerpt from the `Connect` program, enhanced to handle these conditions.

Example 14-4. ConnectFriendly.java

```
public class ConnectFriendly {
 public static void main(String[] argv) {
  String serverName = argv.length == 1 ? argv[0] : "localhost";
  int tcpPort = 80;
  try (Socket sock = new Socket(serverName, tcpPort)) {

   /* If we get here, we can read and write on the socket. */
   System.out.println(" *** Connected to " + serverName  + " ***");

   /* Do some I/O here... */

  } catch (UnknownHostException e) {
   System.err.println(serverName + " Unknown host");
   System.exit(1);
  } catch (NoRouteToHostException e) {
   System.err.println(serverName + " Unreachable" );
   System.exit(1);
  } catch (ConnectException e) {
   System.err.println(serverName + " Connection Refused");
   System.exit(1);
  } catch (java.io.IOException e) {
   System.err.println(serverName + ' ' + e);
   System.exit(1);
  }
 }
}
```

Some of the messages can be easily demonstrated:

```
$ java main/src/main/java/network/ConnectFriendly.java
 *** Connected to localhost ***
$ java main/src/main/java/network/ConnectFriendly.java darwinsys.com
```

```
 *** Connected to darwinsys.com ***
$ java main/src/main/java/network/ConnectFriendly.java adventure.plugh
adventure.plugh Unknown host
$ java main/src/main/java/network/ConnectFriendly.java 10.11.12.13
^C  # Interrupted as it hung for a long time.
$
```

14.5 Reading and Writing Textual Data

Problem

Having connected, you want to transfer textual data.

Solution

Construct a `BufferedReader` or `PrintWriter` from the socket's `getInputStream()` or `getOutputStream()`.

Discussion

The `Socket` class has methods that allow you to get an `InputStream` or `OutputStream` to read from or write to the socket. It has no method to fetch a `Reader` or `Writer`, partly because some network services are limited to ASCII, but mainly because the `Socket` class was decided on before there were `Reader` and `Writer` classes. You can always create a `Reader` from an `InputStream` or a `Writer` from an `OutputStream` using the conversion classes. This is the paradigm for the two most common forms:

```
BufferedReader is = new BufferedReader(
   new InputStreamReader(sock.getInputStream( )));
PrintWriter os = new PrintWriter(sock.getOutputStream( ), true);
```

Example 14-5 reads a line of text from the daytime service, which is offered by full-fledged TCP/IP suites (such as those included with most Unixes). You don't have to send anything to the `Daytime` server; you simply connect and read one line. The server writes one line containing the date and time and then closes the connection. Running it looks like the following code. I started by getting the current date and time on the local host, then ran the `DaytimeText` program to see the date and time on one of my Unix servers:

```
C:\javasrc\network>date
Current date is Sun 01-23-2000
Enter new date (mm-dd-yy):
C:\javasrc\network>time
Current time is  1:13:18.70p
Enter new time:
C:\javasrc\network>java network.DaytimeText myserver
Time on myserver is Sun Jan 26 13:14:34 2025
```

The code is in Example 14-5.

Example 14-5. main/src/main/java/network/DaytimeText.java

```
public class DaytimeText {
  public static final short TIME_PORT = 13;

  public static void main(String[] argv) {
    String server_name = argv.length == 1 ? argv[0] : "localhost";

    try (Socket sock = new Socket(server_name,TIME_PORT);
      BufferedReader is = new BufferedReader(new
        InputStreamReader(sock.getInputStream()));) {
      String remoteTime = is.readLine();
      System.out.println("Time on " + server_name + " is " + remoteTime);
    } catch (IOException e) {
      System.err.println(e);
    }
  }
}
```

The second example, shown in Example 14-6, shows both reading and writing on the same socket. The `Echo` server simply echoes back whatever lines of text you send it. It's not a very clever server, but it is a useful one. It helps in network testing and also in testing clients of this type!

The `converse()` method holds a short conversation with the `Echo` server on the named host; if no host is named, it tries to contact `localhost`, a universal alias[3] for the machine the program is running on.

Example 14-6. main/src/main/java/network/EchoClientOneLine.java

```
public class EchoClientOneLine {
  /** What we send across the net */
  String mesg = "Hello across the net";

  public static void main(String[] argv) {
    if (argv.length == 0)
      new EchoClientOneLine().converse("localhost");
    else
      new EchoClientOneLine().converse(argv[0]);
  }

  /** Hold one conversation across the net */
  protected void converse(String hostName) {
    try (Socket sock = new Socket(hostName, 7);) { // echo server.
      BufferedReader is = new BufferedReader(new
        InputStreamReader(sock.getInputStream()));
      PrintWriter os = new PrintWriter(sock.getOutputStream(), true);
      // Do the CRLF ourself since println appends only a \r on
      // platforms where that is the native line ending.
      os.print(mesg + "\r\n"); os.flush();
      String reply = is.readLine();
      System.out.println("Sent \"" + mesg  + "\"");
      System.out.println("Got  \"" + reply + "\"");
    } catch (IOException e) {
      System.err.println(e);
    }
  }
}
```

It might be a good exercise to isolate the reading and writing code from this method into a `NetWriter` class, possibly subclassing `PrintWriter` and adding the `\r\n` and the flushing.

14.6 Reading and Writing Binary or Serialized Data

Problem

Having connected, you want to transfer binary data, either raw binary data or serialized Java objects.

Solution

For plain binary data, construct a `DataInputStream` or `DataOutputStream` from the socket's `getInputStream()` or `getOutputStream()`. For serialized Java object data, construct an `ObjectInputStream` or `ObjectOutputStream`.

Discussion

The simplest paradigm for reading/writing on a socket is this:

```
DataInputStream is = new DataInputStream(sock.getInputStream());
DataOutputStream is = new DataOutputStream(sock.getOutputStream( ));
```

If the volume of data might be large, insert a buffered stream for efficiency. The paradigm is this:

```
DataInputStream is = new DataInputStream(
    new BufferedInputStream(sock.getInputStream( )));
DataOutputStream is = new DataOutputStream(
    new BufferedOutputStream(sock.getOutputStream( )));
```

The program in Example 14-7 uses another standard service that gives out the time, this one as a binary integer representing the number of seconds since 1900. Because the Java `Date` class base (like Unix time) is 1970, we convert the time base by subtracting the difference between 1970 and 1900. When I used this exercise in a course, most of the students wanted to *add* this time difference, reasoning that 1970 is later. But if you think clearly, you'll see that there are fewer seconds between 1999 and 1970 than there are between 1999 and 1900, so subtraction gives the correct number of seconds. And

because the `Date` constructor needs milliseconds, we multiply the number of seconds by 1,000.

The time difference is the number of years multiplied by 365, plus the number of leap days between the two dates (in the years 1904, 1908, . . . , 1968)—19 days.

The integer that we read from the server is a C-language `unsigned int`. But Java doesn't provide an unsigned integer type; normally when you need an unsigned number, you use the next-larger integer type, which would be `long`. But Java also doesn't give us a method to read an unsigned integer from a data stream. The `DataInputStream` method `readInt()` reads Java-style signed integers. There are `readUnsignedByte()` methods and `readUnsignedShort()` methods, but no `readUnsignedInt()` method. Accordingly, we synthesize the ability to read an unsigned `int` (which must be stored in a `long`, or else you'd lose the signed bit and be back where you started from) by reading unsigned bytes and reassembling them using Java's bit-shifting operators. At the end of the code, we use the new date/time API (see Chapter 6) to construct and print a `LocalDateTime` object to show the current date and time on the local (client) machine, taking the time zone difference from GMT into account:

```
$ java TimeClient.java  localhost
Remote time is 3930648456
BASE_DIFF is 2208988800
Time diff == 1721659656
Time on localhost is 2024-07-22T10:47:36
Local date/time = 2024-07-22T10:47:36.848233148
$ java TimeClient.java darwinsys.com
Remote time is 3930648485
BASE_DIFF is 2208988800
Time diff == 1721659685
Time on darwinsys.com is 2024-07-22T10:48:05
Local date/time = 2024-07-22T10:48:05.438931478
```

Looking at the output, you can see that the server and localhost agree on the time, within a second. That confirms the date calculation code in Example 14-7.

Example 14-7. main/src/main/java/network/TimeClient.java

```
public class TimeClient {
 /** The TCP port for the binary time service. */
 public static final short TIME_PORT = 37;
```

```
  /** Seconds between 1970, the time base for dates and times
   * Factors in leap years (up to 2100), hours, minutes, and seconds.
   * Subtract 1 day for 1900, add in 1/2 day for 1969/1970.
   */
  protected static final long BASE_DAYS =
    (long)((1970-1900)*365 + (1970-1900-1)/4);

  /* Seconds since 1970 */
  public static final long BASE_DIFF = (BASE_DAYS * 24 * 60 * 60);

  public static void main(String[] argv) {
    String hostName;
    if (argv.length == 0)
      hostName = "localhost";
    else
      hostName = argv[0];

    try (Socket sock = new Socket(hostName,TIME_PORT);) {
      DataInputStream is = new DataInputStream(new
        BufferedInputStream(sock.getInputStream()));
      // Read 4 bytes from the network, unsigned.
      // Do it yourself; there is no readUnsignedInt().
      // Long is 8 bytes on Java, but we are using the
      // existing time protocol, which uses 4-byte ints.
      long remoteTime = (
        ((long)(is.readUnsignedByte()) << 24) |
        ((long)(is.readUnsignedByte()) << 16) |
        ((long)(is.readUnsignedByte()) <<  8) |
        ((long)(is.readUnsignedByte()) <<  0));
      System.out.println("Remote time is " + remoteTime);
      System.out.println("BASE_DIFF is " + BASE_DIFF);
      System.out.println("Time diff == " + (remoteTime - BASE_DIFF));
      Instant time = Instant.ofEpochSecond(remoteTime - BASE_DIFF);
      LocalDateTime d = LocalDateTime.ofInstant(time, ZoneId.systemDefault());
      System.out.println("Time on " + hostName + " is " + d.toString());
      System.out.println("Local date/time = " + LocalDateTime.now());
    } catch (IOException e) {
      System.err.println(e);
    }
  }
}
```

Object serialization is the ability to convert in-memory objects to an external form that can be sent serially (as a byte stream). To read or write Java objects via serialization, you need only construct an `ObjectInputStream` or `ObjectOutputStream` from an

`InputStream` or `OutputStream`; in this case, the socket's `getInputStream()` or `getOutputStream()`.

This program (and its server) provides a service that isn't a standard part of the TCP/IP stack; it's a service I made up as a demo. The server for this service is introduced in Recipe 15.3. The client code in Example 14-8 is quite similar to the `DaytimeBinary` program in the previous recipe, but the server sends us a `LocalDateTime` object already constructed. Example 14-8 shows the portion of the client code that differs from Example 14-7.

Example 14-8. main/src/main/java/network/DaytimeObject.java

```
try (Socket sock = new Socket(hostName, TIME_PORT);) {
 ObjectInputStream is = new ObjectInputStream(new
  BufferedInputStream(sock.getInputStream()));

 // Read and validate the Object
 Object o = is.readObject();
 if (o == null) {
  System.err.println("Read null from server!");
 } else if ((o instanceof LocalDateTime d)) {
  System.out.println("Time on " + hostName + " is " + d);
 } else {
  throw new IllegalArgumentException(
   "Wanted LocalDateTime, got %s, a %s".formatted(
    o, o.getClass()));
 }
```

I ask the operating system for the date and time, and then I run the program, which prints the date and time on a networked machine named aragorn:

```
$ date
Thu Dec 26 09:29:02 EST 2025
C:\javasrc\network>java network.DaytimeObject aragorn
Time on aragorn is 2025-12-26T09:29:05.227397
C:\javasrc\network>
```

Again, the results agree within a few seconds.

14.7 Postcards of the Internet: Using UDP Datagrams

Problem

You need to use a datagram connection (UDP) instead of a stream connection (TCP).

Solution

Use `DatagramSocket` and `DatagramPacket`.

Discussion

Datagram network traffic is a kindred spirit to the underlying packet-based Ethernet and IP (Internet Protocol) layers. Unlike a stream-based connection such as TCP, datagram transports like UDP transmit each *packet*, or chunk of data, as a single entity with no necessary relation to any other.[4] A common analogy is that TCP is like talking on the telephone, whereas UDP is like sending postcards or maybe fax messages.

The differences show up most in error handling. Packets can, like postcards, go astray. When was the last time the postman rang your bell to tell you that the post office had lost one of several postcards it was supposed to deliver to you? That's not going to happen, because the post office doesn't keep track of postcards. On the other hand, when you're talking on the phone and there's a noise burst—like somebody yelling in the room, or even a signal dropout—you notice the failure in real time, and you can ask the person at the other end to repeat what they just said.

With a stream-based connection like a TCP socket, the network transport layer handles errors for you: it asks the other end to retransmit. With a datagram transport such as UDP, you have to handle retransmission yourself. It's kind of like numbering the postcards you send so that you can go back and resend any that don't arrive—a good excuse to return to your vacation spot, perhaps.

Another difference is that datagram transmission preserves message boundaries. That is, if you write 20 bytes and then write 10 bytes when using TCP, the program reading from the other end will not know if you wrote one chunk of 30 bytes, two chunks of 15, or even 30 individual characters. With a `DatagramSocket`, you construct a `DatagramPacket` object for each buffer, and its contents are sent as a *single* entity over

the network; its contents will not be mixed together with the contents of any other buffer. The `DatagramPacket` object has methods like `getLength()` and `setPort()`.

IAN'S BASIC STEPS: UDP CLIENT

UDP is a bit more involved, so I'll list the basic steps for generating a UDP client:

1. Create a `DatagramSocket` with no arguments (the form that takes two arguments is used on the server).
2. Optionally `connect()` the socket to an `InetAddress` (see Recipe 14.3) and port number.
3. Create one or more `DatagramPacket` objects; these are wrappers around a byte array that contains data you want to send and is filled in with data you receive.
4. If you did not `connect()` the socket, provide the `InetAddress` and port when constructing the `DatagramPacket`.
5. Set the packet's length and use `sock.send(packet)` to send data to the server.
6. Use `sock.receive()` to retrieve data.

Note that the optional `connect()` call only establishes the default address for packets; UDB is still a "connectionless" protocol.

So why would we even use UDP? UDP has a lot less overhead than TCP, which can be particularly valuable when sending huge amounts of data over a reliable local network or a few hops on the internet. Over long-haul networks, TCP is generally preferred because TCP handles retransmission of lost packets for you. And obviously, if preserving record boundaries makes your life easier, that may be a reason for considering UDP. UDP is also the way to perform Multicast (send to many receivers simultaneously), though Multicast is out of scope for this discussion.

Example 14-9 is a short program that connects via UDP to the `Daytime` date and time server used in Recipe 14.5. Because UDP has no real notion of connection, the client typically initiates the conversation, which sometimes means sending an empty packet; the UDP server uses the address information it gets from that to return its response.

Example 14-9. main/src/main/java/network/DaytimeUDP.java

```
public class DaytimeUDP {
  /** The UDP port number */
  public final static int DAYTIME_PORT = 13;
```

```
  /** A buffer plenty big enough for the date string */
  protected final static int PACKET_SIZE = 100;

  /** The main program that drives this network client.
   * @param argv[0] hostname, running daytime/udp server
   */
  public static void main(String[] argv) throws IOException {
    if (argv.length < 1) {
      System.err.println("usage: java DayTimeUDP host");
      System.exit(1);
    }
    String host = argv[0];
    InetAddress servAddr = InetAddress.getByName(host);
    DatagramSocket sock = new DatagramSocket();
    byte[] buffer = new byte[PACKET_SIZE];

    // The UDP packet we will send and receive
    DatagramPacket packet = new DatagramPacket(
      buffer, PACKET_SIZE, servAddr, DAYTIME_PORT);

    /* Send empty max-length (-1 for null byte) packet to server */
    packet.setLength(PACKET_SIZE-1);
    sock.send(packet);
    System.out.println("Sent request");

    // Receive a packet and print it.
    sock.receive(packet);
    System.out.println("Got packet of size " + packet.getLength());
    System.out.print("Date on " + host + " is " +
      new String(buffer, 0, packet.getLength()));

    sock.close();
  }
}
```

I'll run it to my Unix box just to be sure that it works:

```
$
$ java network.DaytimeUDP aragorn
Sent request
Got packet of size 26
Date on aragorn is Fri Dec 12 09:29:05 2025
$
```

14.8 URI, URL, or URN?

Problem

Having heard these terms, you want to know the difference between a URI, URL, and URN.

Solution

Read on. Or see the Javadoc for *java.net.URI*.

Discussion

A URL is the traditional name for a network address consisting of a scheme (like `https`) and an address (site name) and resource or pathname. But there are three distinct terms in all:

- URI (Uniform Resource Identifier)
- URL (Uniform Resource Locator)
- URN (Uniform Resource Name)

A discussion near the end of the Java documentation for the `URI` class explains the relationship among URI, URL, and URN. URIs form the set of all identifiers. URLs and URNs are subsets. URIs are the most general; a URI is parsed for basic syntax without regard to the scheme, if any, that it specifies, and it need not refer to a particular server. A URL includes a hostname, scheme, and other components; the string is parsed according to rules for its scheme. When you construct a URL, an `InputStream` is created automatically. URNs name resources but do not explain how to locate them; typical examples of URNs that you will have seen include `mailto:` and occasionally `tel:` references.

The main operations provided by the `URI` class are normalization (removing extraneous path segments including `..`) and relativization (this should be called "making relative," but somebody wanted a single word to make a method name). A `URI` object does not have any methods for opening the URI; for that, you would normally use a string representation of the URI to construct a URL object, like so:

```
URL x = new URL(theURI.toString( ));
```

The program in Example 14-10 shows examples of normalizating, making relative, and constructing a URL from a URI.

Example 14-10. main/src/main/java/netweb/URIDemo.java

```
public class URIDemo {
  public static void main(String[] args)
  throws URISyntaxException, MalformedURLException {

    // Start with un-normalized example
    URI u = new URI("https://darwinsys.com/java/../openbsd/../index.html");
    System.out.println("Raw: " + u);
    URI normalized = u.normalize();
    System.out.println("Normalized: " + normalized);
    final URI BASE = new URI("https://darwinsys.com");
    System.out.println("Relativized to " + BASE + ": " + BASE.relativize(u));

    // A URL is a type of URI
    URL url = URI.create(normalized.toString()).toURL();
    System.out.println("URL: " + url);

    // Demo of non-URL but valid URI
    URI uri = new URI("bean:WonderBean");
    System.out.println(uri);
  }
}
```

Running it produces this output:

```
$ java URIDemo.java
Raw: https://darwinsys.com/java/../openbsd/../index.html
Normalized: https://darwinsys.com/index.html
Relativized to https://darwinsys.com: index.html
URL: https://darwinsys.com/index.html
bean:WonderBean
$
```

14.9 Program: Sockets-Based Chat Client

This program is a very simple chat program. You can't talk to Google Chat, Instagram, or X/Twitter (never mind ancient chat-only protocols like ICQ or AIM) with it, because they

each use their own protocol.[5] Rather, this program simply writes to and reads from a server. The server for this will be presented in Chapter 15. How does it look when you run it? Figure 14-2 shows me chatting all by myself one day.

The code is reasonably self-explanatory. We read from the remote server in a thread to make the input and the output run without blocking each other; this is discussed in Chapter 11. The techniques for reading and writing have been discussed throughout this chapter. The code is not shown here but is in *javasrc* at *main/src/main/java/chat/ChatClient.java* and online at my GitHub repository.

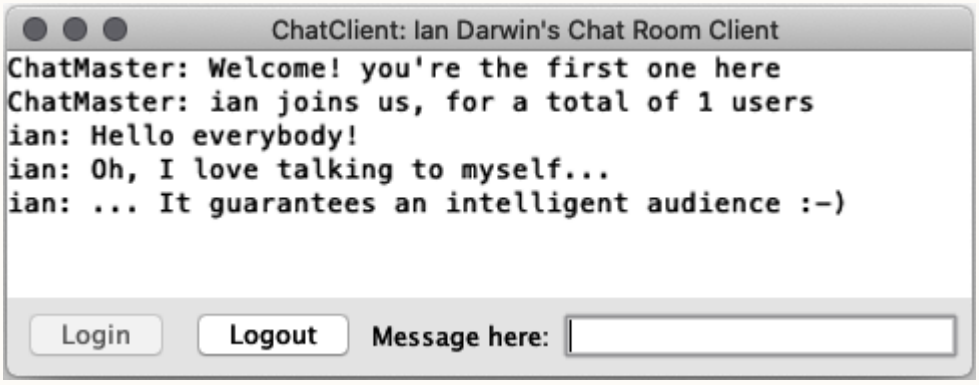


Figure 14-2. Chat client in action

See Also

There are many better-structured ways to write a chat client, including WebSockets, XMPP, RMI, and JMS. XMPP is an open protocol for chat, with tools for Java and many other languages. RMI is Java's RPC interface and for many years was included in both Java SE and Java EE; it is not described in this edition of this book, but you can find the RMI chapter from previous editions on my website. The other technologies are part of Jakarta Enterprise, so you should refer to their documentation. If your communication goes over the public internet, you do need to encrypt your socket connection, so check out Sun's Java Secure Socket Extension (JSSE). If you took my earlier advice and used the standard HTTP protocol, you can encrypt the conversation just by changing the URL from `http` to `https`.

For a good overview of network programming from the C programmer's point of view, see the late W. Richard Stevens's *Unix Network Programming* (Prentice Hall). Despite the book's name, it's mainly about socket and TCP/IP/UDP programming and covers all parts of the (Unix) networking API and protocols such as TFTP in amazing detail.

1 Maximum possible IPv6 addresses = 2^{128} = 3.402824×10^{38}. Amount of dry land is approximately 148.94 million km^2 or 148.94×10^{12} m^2. Area of one smartphone (Pixel 6A) about 0.01092836 m^2.

Divide the total area by the area of one unit: Number of smartphones≈$148.94\times10^{12}/0.01092836 \approx 1.363\times10^{16}$. Approximate ratio: $10^{38}/10^{16} = 10^{22}$, or 22 orders of magnitude.

2 The location where it is looked up varies. It might be in a file named */etc/services* on Unix; in the *services* file in a subdirectory like *\Windows\System32\drivers\etc* in Microsoft Windows; in a centralized registry such as the Domain Name Service, or Sun's Network Information Services (NIS, formerly YP); or in some other platform- or network-dependent location.

3 It used to be universal, when most networked systems were administered by full-time system admins who had been trained or served an apprenticeship. Today many machines on the internet don't have `localhost` configured properly.

4 The UDP packet may need to be fragmented by some networks, but this is not germane to us at the UDP level, because it will reassemble the network packets into our single-entity UDP packet at the other end.

5 For an open source program that does provide an IM service to let you talk to various chats from a single program, check out Pidgin or Jabber.

Previous chapter
‹ 13. Machine Learning/Artificial Intelligence

Next chapter
15. Server-Side Java ›